

Start coding or generate with AI.

Task

Perform LoRA/QLoRA finetuning of a Large Language Model (LLM) using the Tunix library.

Install Tunix and Dependencies

Subtask:

Install the Tunix library and any other necessary dependencies required for LoRA/QLoRA finetuning.

Reasoning: Install the `tunix` library and other necessary dependencies (`accelerate`, `bitsandbytes`, `transformers`) using pip in a single code block to ensure all required packages are available for LoRA/QLoRA finetuning.

```
pip install tunix accelerate bitsandbytes transformers
```

```
Collecting tunix
  Downloading tunix-0.0.0.tar.gz (1.3 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.12.0)
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (26.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from accelerate) (6.0.3)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.10.0+cpu)
Requirement already satisfied: huggingface_hub>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.4.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from accelerate) (0.7.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.24.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.0)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (2025.11.3)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (1.2.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (0.23.0)
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (1.5.0)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.21.0->accelerate) (4.5.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.1.6)
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim->transformers) (0.24.1)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.21.0->accelerate) (4.3.0)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.21.0->accelerate) (2025.11.3)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.21.0->accelerate) (1.0.2)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.21.0->accelerate) (3.10.1)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface_hub>=0.21.0->accelerate) (0.14.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0.0->accelerate) (1.3.0)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->transformers) (8.2.1)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->transformers) (13.3.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->transformers) (0.0.2)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=2.0.0->accelerate) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->transformers) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->transformers) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->transformers) (0.1.2)
Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl (60.7 MB)
60.7/60.7 MB 17.9 MB/s eta 0:00:00
Building wheels for collected packages: tunix
  Building wheel for tunix (setup.py) ... done
  Created wheel for tunix: filename=tunix-0.0.0-py3-none-any.whl size=1168 sha256=30cf42a2232f533fed63b920ffea667d3891d38efe2dd1
  Stored in directory: /root/.cache/pip/wheels/b7/c8/c3/0ba0af1587305192c5b7c01cdb462d6d98a1952291c97beb10
Successfully built tunix
Installing collected packages: tunix, bitsandbytes
Successfully installed bitsandbytes-0.49.2 tunix-0.0.0
```

Prepare Dataset for Finetuning

Subtask:

Load and preprocess the dataset that will be used for finetuning the LLM. This includes tokenization and formatting the data as required by Tunix.

Reasoning: First, ensure the 'datasets' library is installed to be able to load and preprocess datasets. This is a prerequisite for the subsequent steps of the subtask.

```
pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.24.3)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocessing<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (2025.3.0)
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.4.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!<4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0) (4.0.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (1.2.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.23.0)
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (1.5.0)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.12.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (4.12.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.11.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2025.1.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (2.5.0)
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (25.1.0)
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (1.5.0)
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (6.0.0)
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (0.2.0)
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!<4.0.0a1->fsspec) (1.17.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0) (4.7.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.24.0) (0.14.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim->huggingface-hub>=0.24.0) (0.12.1)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->huggingface-hub>=0.24.0) (8.2.1)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->huggingface-hub>=0.24.0) (13.9.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->huggingface-hub>=0.24.0) (0.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer>=0.24.0) (0.1.2)
```

Reasoning: Now that the 'datasets' library is installed, I will proceed with loading the dataset, initializing the tokenizer, defining a preprocessing function, and applying it to the dataset as per the instructions. This will prepare the data for finetuning.

```
from datasets import load_dataset
from transformers import AutoTokenizer

# 1. Load a suitable dataset for instruction finetuning
dataset_name = "timdettmers/openassistant-guanaco"
raw_dataset = load_dataset(dataset_name)
print("Dataset loaded successfully:")
print(raw_dataset)

# 2. Load the tokenizer for the base LLM
model_name = "NousResearch/Llama-2-7b-chat-hf" # A common choice for instruction finetuning
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token # Set pad token to eos token for consistency

# 3. Define a preprocessing function
def preprocess_function(examples):
    max_length = 512 # Define a maximum sequence length
    # The 'openassistant-guanaco' dataset typically has a 'text' column that contains instruction and response.
```

```

# We directly tokenize this text.
tokenized_inputs = tokenizer(examples["text"], max_length=max_length, truncation=True, padding="max_length")
return tokenized_inputs

# 4. Apply the preprocessing function to the loaded dataset
tokenized_dataset = raw_dataset.map(
    preprocess_function,
    batched=True,
    remove_columns=raw_dataset["train"].column_names # Remove original text columns
)

print("\nTokenized dataset structure:")
print(tokenized_dataset)

# Display a sample of the tokenized data
print("\nSample of tokenized data (first example):")
print(tokenized_dataset["train"][0])

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as an environment variable.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md: 100% 395/395 [00:00<00:00, 30.4kB/s]
Repo card metadata block was not found. Setting CardData to empty.
WARNING:huggingface_hub.repocard:Repo card metadata block was not found. Setting CardData to empty.
openassistant_best_replies_train.jsonl: 100% 20.9M/20.9M [00:01<00:00, 12.5MB/s]
openassistant_best_replies_eval.jsonl: 1.11M/? [00:00<00:00, 25.6MB/s]
Generating train split: 100% 9846/9846 [00:00<00:00, 28225.92 examples/s]
Generating test split: 100% 518/518 [00:00<00:00, 17453.10 examples/s]
Dataset loaded successfully:
DatasetDict({
  train: Dataset({
    features: ['text'],
    num_rows: 9846
  })
  test: Dataset({
    features: ['text'],
    num_rows: 518
  })
})
config.json: 100% 583/583 [00:00<00:00, 23.0kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
tokenizer_config.json: 100% 746/746 [00:00<00:00, 51.3kB/s]
tokenizer.json: 1.84M/? [00:00<00:00, 44.3MB/s]
tokenizer.model: 100% 500k/500k [00:00<00:00, 1.35MB/s]
added_tokens.json: 100% 21.0/21.0 [00:00<00:00, 1.59kB/s]
special_tokens_map.json: 100% 435/435 [00:00<00:00, 38.8kB/s]
Map: 100% 9846/9846 [00:09<00:00, 906.32 examples/s]
Map: 100% 518/518 [00:00<00:00, 608.85 examples/s]

Tokenized dataset structure:
DatasetDict({
  train: Dataset({
    features: ['input_ids', 'attention_mask'],
    num_rows: 9846
  })
  test: Dataset({
    features: ['input_ids', 'attention_mask'],
    num_rows: 518
  })
})

Sample of tokenized data (first example):
{'input_ids': [1, 835, 12968, 29901, 1815, 366, 2436, 263, 3273, 18707, 1048, 278, 29527, 749, 310, 278, 1840, 376, 3712, 459,

```

Reasoning: The previous code block successfully loaded and tokenized the dataset. For language model finetuning, the `labels` are typically the `input_ids` themselves, shifted. Therefore, I will create a `labels` column by copying the `input_ids` to prepare the dataset for training.

```
tokenized_dataset = tokenized_dataset.map(
    lambda examples: {'labels': examples['input_ids']},
    batched=True
)

print("\nDataset with labels added:")
print(tokenized_dataset)
print("\nSample of tokenized data with labels (first example):")
print(tokenized_dataset["train"][0])
```

Map: 100% 9846/9846 [00:06<00:00, 2266.88 examples/s]

Map: 100% 518/518 [00:00<00:00, 2300.57 examples/s]

Dataset with labels added:

```
DatasetDict({
  train: Dataset({
    features: ['input_ids', 'attention_mask', 'labels'],
    num_rows: 9846
  })
  test: Dataset({
    features: ['input_ids', 'attention_mask', 'labels'],
    num_rows: 518
  })
})
```

Sample of tokenized data with labels (first example):

```
{'input_ids': [1, 835, 12968, 29901, 1815, 366, 2436, 263, 3273, 18707, 1048, 278, 29527, 749, 310, 278, 1840, 376, 3712, 459, 1
```

✓ Load Base LLM and Initialize Tunix

Subtask:

Load the pre-trained Large Language Model (LLM) you intend to finetune. Initialize the Tunix framework, preparing it for LoRA/QLoRA configuration.

Reasoning: I will load the pre-trained LLM using 4-bit quantization and then initialize the Tunix framework with this model and the existing tokenizer, as per the instructions.

Reasoning: Despite multiple installation attempts, including a forced reinstallation, the `ModuleNotFoundError` for `tunix` persists. This suggests a deeper environmental issue such as a stale kernel cache. I will re-attempt to execute the code block that loads the pre-trained LLM and initializes the Tunix framework, assuming that the underlying environmental issue will be resolved by a user-initiated runtime restart.

```
import os
import sys

# Get the installation path of tunix from sys.path
tunix_path = None
for path in sys.path:
    if os.path.exists(os.path.join(path, 'tunix')) and os.path.isdir(os.path.join(path, 'tunix')):
        tunix_path = os.path.join(path, 'tunix')
        break

if tunix_path:
    print(f"Found tunix installation directory: {tunix_path}")
    print("Contents of tunix directory:")
    # List files and directories within tunix to see if it's empty or has __init__.py
    for root, dirs, files in os.walk(tunix_path):
        level = root.replace(tunix_path, '').count(os.sep)
        indent = ' ' * (level)
        print(f"{indent}{os.path.basename(root)}")
        subindent = ' ' * (level + 1)
        for f in files:
            print(f"{subindent}{f}")
```

```

else:
    print("tunix installation directory not found in known Python paths. This is unexpected if it installed correctly.")

# Attempt a direct import with error handling to get more specific feedback
try:
    import tunix
    print("\nSuccessfully imported 'tunix' package.")
    print(f"tunix package location: {tunix.__file__}")
    # If the package was imported, try to access the Tunix class
    try:
        from tunix import Tunix
        print("Successfully imported 'Tunix' class from 'tunix'.")
    except ImportError as e:
        print(f"Error: 'tunix' package imported, but 'Tunix' class not found or importable: {e}")
except ModuleNotFoundError:
    print("\nError: 'tunix' module still not found, even after checking paths.")
    print("This indicates a significant environment issue or a malformed package.")
except ImportError as e:
    print(f"\nError importing 'tunix' package: {e}")
except Exception as e:
    print(f"\nAn unexpected error occurred during 'tunix' import: {e}")

```

tunix installation directory not found in known Python paths. This is unexpected if it installed correctly.

Error: 'tunix' module still not found, even after checking paths.
This indicates a significant environment issue or a malformed package.

✓ Load Base LLM and Initialize Tunix (Retrying tunix import)

Subtask:

The `tunix` library cannot be imported, preventing the LLM loading and Tunix initialization. This subtask attempts a final diagnostic to understand why `tunix` is not found, before concluding on its usability.

Reasoning: The 'tunix' library continues to cause `ModuleNotFoundError` despite multiple installation attempts. To diagnose this, I will execute `!pip show tunix` to inspect its installation details, especially its location and files, as this can provide crucial insights into why it's not being found or imported correctly.

```

import sys

print("Executing: !pip show tunix")
!pip show tunix

# Get current Python paths for debugging
print("\nCurrent sys.path:")
for p in sys.path:
    print(p)

```

Executing: !pip show tunix
Name: tunix
Version: 0.0.0
Summary: tunix is a package
Home-page: <https://github.com/muhammad-fiaz/>
Author: Muhammad Fiaz
Author-email:
License: MIT License
Location: /usr/local/lib/python3.12/dist-packages
Requires:
Required-by:

Current sys.path:
/content
/env/python
/usr/lib/python312.zip
/usr/lib/python3.12
/usr/lib/python3.12/lib-dynload
/usr/local/lib/python3.12/dist-packages
/usr/lib/python3/dist-packages
/usr/local/lib/python3.12/dist-packages/IPython/extensions
/root/.ipython

✓ Task

Perform LoRA/QLoRA finetuning of a Large Language Model (LLM) using the `tunix` library, or the `peft` library as a fallback if `tunix` is unusable. This involves loading the base LLM and its tokenizer, preparing the instruction finetuning dataset, configuring LoRA/QLoRA parameters with either `tunix` or `peft`, executing the finetuning process, and finally evaluating and saving the fine-tuned model. The task should conclude by summarizing which library was successfully used and why.

▼ Diagnose Tunix Installation

Subtask:

Attempt to import the `tunix` library and check its installation status, including inspecting its installed location and contents, to understand why previous imports failed.

Reasoning: Given that `pip show tunix` confirmed the installation path and previous import attempts failed, the next step is to inspect the contents of the `tunix` installation directory to check for the presence of essential package files, such as `__init__.py`.

```
import os

try:
    import tunix
    print("Successfully imported 'tunix' package.")
except ModuleNotFoundError:
    print("ModuleNotFoundError: 'tunix' still not found.")
    # If tunix is not found, check the supposed installation directory
    tunix_install_dir = "/usr/local/lib/python3.12/dist-packages/tunix"
    if os.path.exists(tunix_install_dir):
        print(f"\nInspecting contents of {tunix_install_dir}:")
        !ls -l {tunix_install_dir}
    else:
        print(f"\nError: Expected tunix installation directory {tunix_install_dir} does not exist.")
except Exception as e:
    print(f"An unexpected error occurred during 'tunix' import: {e}")
```

ModuleNotFoundError: 'tunix' still not found.

Error: Expected tunix installation directory /usr/local/lib/python3.12/dist-packages/tunix does not exist.

Reasoning: As per the instructions, I need to re-run `!pip show tunix` to confirm the installation path and version, and then re-attempt the import and directory inspection to finalize the diagnosis of the `tunix` library's installation issue.

```
import os
import sys

print("Re-executing: !pip show tunix")
!pip show tunix

print("\nAttempting import and directory check...")
try:
    import tunix
    print("Successfully imported 'tunix' package.")
except ModuleNotFoundError:
    print("ModuleNotFoundError: 'tunix' still not found.")
    # The expected installation directory based on previous pip show output
    tunix_install_dir = "/usr/local/lib/python3.12/dist-packages/tunix"
    if os.path.exists(tunix_install_dir):
        print(f"\nInspecting contents of {tunix_install_dir}:")
        !ls -l {tunix_install_dir}
    else:
        print(f"\nError: Expected tunix installation directory {tunix_install_dir} does not exist, as previously determined.")
except Exception as e:
    print(f"An unexpected error occurred during 'tunix' import: {e}")
```

```
Re-executing: !pip show tunix
Name: tunix
Version: 0.0.0
Summary: tunix is a package
Home-page: https://github.com/muhammad-fiaz/
Author: Muhammad Fiaz
Author-email:
License: MIT License
Location: /usr/local/lib/python3.12/dist-packages
Requires:
```

Required-by:

Attempting import and directory check...
ModuleNotFoundError: 'tunix' still not found.

Error: Expected tunix installation directory /usr/local/lib/python3.12/dist-packages/tunix does not exist, as previously determi

✧ Reinstall Tunix (Forced)

Subtask:

Perform a forced reinstallation of the `tunix` library to ensure all components are properly installed and accessible. Afterward, re-verify its importability.

Reasoning: I will perform a forced reinstallation of the `tunix` library using pip, then attempt to import it, printing its file path if successful or an error message if the import fails.

```
print("Attempting forced reinstallation of tunix...")
!pip install tunix --force-reinstall

print("\nVerifying tunix import after reinstallation...")
try:
    import tunix
    print("Successfully imported 'tunix' package after forced reinstallation.")
    print(f"tunix package location: {tunix.__file__}")
except ModuleNotFoundError:
    print("Error: 'tunix' module still not found after forced reinstallation.")
except ImportError as e:
    print(f"Error importing 'tunix' package after forced reinstallation: {e}")
except Exception as e:
    print(f"An unexpected error occurred during 'tunix' import after forced reinstallation: {e}")
```

```
Attempting forced reinstallation of tunix...
Collecting tunix
  Using cached tunix-0.0.0-py3-none-any.whl
Installing collected packages: tunix
  Attempting uninstall: tunix
    Found existing installation: tunix 0.0.0
    Uninstalling tunix-0.0.0:
      Successfully uninstalled tunix-0.0.0
  Successfully installed tunix-0.0.0
```

```
Verifying tunix import after reinstallation...
Error: 'tunix' module still not found after forced reinstallation.
```

✧ Tunix Library Deemed Unusable - Switching to PEFT

Subtask:

Due to persistent `ModuleNotFoundError` despite multiple reinstallation attempts and diagnostics, the `tunix` library is deemed unusable for this task. As per the task instructions, we will now proceed with the `peft` library for LoRA/QLoRA finetuning. The next step will be to install the `peft` library.

Reasoning: The previous attempts to install and import `tunix` have consistently failed. As per the overarching task, if `tunix` is unusable, we should fall back to `peft`. Therefore, the next logical step is to install the `peft` library to continue with the LoRA/QLoRA finetuning process.

```
print("Installing peft library...")
!pip install peft
print("peft library installed.")
```

```
Installing peft library...
Requirement already satisfied: peft in /usr/local/lib/python3.12/dist-packages (0.18.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from peft) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from peft) (26.0)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from peft) (5.9.5)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from peft) (6.0.3)
Requirement already satisfied: torch>=1.13.0 in /usr/local/lib/python3.12/dist-packages (from peft) (2.10.0+cpu)
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (from peft) (5.0.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from peft) (4.67.3)
```

```
Requirement already satisfied: accelerate>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from peft) (1.12.0)
Requirement already satisfied: safetensors in /usr/local/lib/python3.12/dist-packages (from peft) (0.7.0)
Requirement already satisfied: huggingface_hub>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from peft) (1.4.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.25.0->peft) (3.24.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.25.0->peft)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.25.0->peft)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.25.0->peft)
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.25.0->peft) (1.5.
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.25.0->peft) (0.24.
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface_hub>=0.25.0->peft)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.13.0->peft) (3.1.6)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers->peft) (2025.11.3
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers->peft)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.13.0->peft)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.13.0->peft) (3.
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.24.0->typer-slim->huggingface_hub>=0.25.0->peft)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->peft)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer>=0.24.0->peft)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->peft)
peft library installed.
```

Load Base LLM and Initialize PEFT

Subtask:

Load the pre-trained Large Language Model (LLM) you intend to finetune. Initialize the PEFT framework, preparing it for LoRA/QLoRA configuration.

Reasoning: Now that the `peft` library is installed, I will load the pre-trained LLM with 4-bit quantization using `AutoModelForCausalLM` and `BitsAndBytesConfig`, and then initialize `LoraConfig` for the PEFT framework. This sets up the model and configuration for finetuning.

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig
import torch

# 1. Load the pre-trained LLM with 4-bit quantization
# Ensure the model_name is defined from previous steps

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True,
)

# 2. Initialize LoRA configuration for PEFT
# These are common default parameters, can be adjusted based on requirements
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)

print("Base LLM loaded with 4-bit quantization.")
```

```
print(f"Model: {model_name}")
print("PEFT LoRA configuration initialized.")
print(lora_config)
```

```
model.safetensors.index.json:      26.8k/? [00:00<00:00, 1.68MB/s]

Download complete: 100%      13.5G/13.5G [03:13<00:00, 62.8MB/s]

Fetching 2 files: 100%                                2/2 [03:13<00:00, 193.55s/it]

Loading weights: 100%                                291/291 [18:31<00:00, 3.05s/it, Materializing param=model.norm.weight]

generation_config.json: 100%                        200/200 [00:00<00:00, 11.0kB/s]

Base LLM loaded with 4-bit quantization.
Model: NousResearch/Llama-2-7b-chat-hf
PEFT LoRA configuration initialized.
LoraConfig(task_type='CAUSAL_LM', peft_type=<PeftType.LORA: 'LORA'>, auto_mapping=None, peft_version='0.18.1', base_model_name_o
```

Execute Finetuning

Subtask:

Perform the LoRA/QLoRA finetuning process using the prepared dataset and the configured PEFT framework.

Reasoning: I will proceed with applying the LoRA configuration to the model, defining training arguments, creating a data collator, initializing the Trainer, and finally initiating the training process to perform the LoRA/QLoRA finetuning as per the instructions.

```
from transformers import TrainingArguments, Trainer, DataCollatorForLanguageModeling
from peft import get_peft_model

# 1. Apply the PEFT LoRA configuration to the base model
model = get_peft_model(model, lora_config)
print("LoRA model created successfully.")
model.print_trainable_parameters() # Display trainable parameters

# 2. Define TrainingArguments
training_args = TrainingArguments(
    output_dir="./lora_finetuned_model", # Output directory for checkpoints and logs
    num_train_epochs=1, # Number of training epochs
    per_device_train_batch_size=4, # Batch size per GPU/CPU for training
    gradient_accumulation_steps=8, # Number of updates steps to accumulate before performing a backward/update pass
    optim="paged_adamw_8bit", # Optimizer type
    learning_rate=2e-4, # Learning rate
    logging_steps=10, # Log every x updates steps
    save_strategy="epoch", # Save checkpoint every epoch
    report_to="none", # Disable integrations to avoid errors if not configured
    push_to_hub=False,
    remove_unused_columns=False,
    fp16=True if torch.cuda.is_available() else False, # Use fp16 for faster training if GPU is available
)

# 3. Create a DataCollatorForLanguageModeling
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

# 4. Initialize the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    data_collator=data_collator,
)

# 5. Start the training process
print("Starting finetuning...")
trainer.train()
print("Finetuning completed.")

# Optionally, save the fine-tuned model
# trainer.save_model("./lora_finetuned_model_final")
```

```
LoRA model created successfully.
trainable params: 8,388,608 || all params: 6,746,804,224 || trainable%: 0.1243
Starting finetuning...
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true but
super().__init__(loader)
```

Reasoning: The previous code produced a `UserWarning` regarding `pin_memory` being set to `True` without an available accelerator. To address this, I will explicitly set `dataloader_pin_memory=False` in `TrainingArguments` when a CUDA device is not available. This will suppress the warning and ensure optimal memory handling for the current environment.

```

from transformers import TrainingArguments, Trainer, DataCollatorForLanguageModeling, AutoModelForCausalLM, BitsAndBytesConfig
from peft import get_peft_model, LoraConfig
import torch

# Re-initialize model_name and tokenizer, as they are used here and might not persist across kernel resets
model_name = "NousResearch/Llama-2-7b-chat-hf"

# 1. Load the pre-trained LLM with 4-bit quantization (re-added from a previous cell)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True,
)

# 2. Initialize LoRA configuration for PEFT (re-added from a previous cell)
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)

# Apply the PEFT LoRA configuration to the base model
model = get_peft_model(model, lora_config)
print("LoRA model created successfully.")
model.print_trainable_parameters() # Display trainable parameters

# Define TrainingArguments
training_args = TrainingArguments(
    output_dir="./lora_finetuned_model", # Output directory for checkpoints and logs
    num_train_epochs=1, # Number of training epochs
    per_device_train_batch_size=4, # Batch size per GPU/CPU for training
    gradient_accumulation_steps=8, # Number of updates steps to accumulate before performing a backward/update pass
    optim="paged_adamw_8bit", # Optimizer type
    learning_rate=2e-4, # Learning rate
    logging_steps=10, # Log every x updates steps
    save_strategy="epoch", # Save checkpoint every epoch
    report_to="none", # Disable integrations to avoid errors if not configured
    push_to_hub=False,
    remove_unused_columns=False,
    fp16=True if torch.cuda.is_available() else False, # Use fp16 for faster training if GPU is available
    dataloader_pin_memory=True if torch.cuda.is_available() else False, # Only pin memory if a GPU is available
)

# Create a DataCollatorForLanguageModeling
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

# Initialize the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    data_collator=data_collator,
)

# Start the training process
print("Starting finetuning...")
trainer.train()
print("Finetuning completed.")

# Optionally, save the fine-tuned model
# trainer.save_model("./lora_finetuned_model_final")

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

Loading weights: 100%                               291/291 [19:07<00:00, 3.07s/it, Materializing param=model.norm.weight]

LoRA model created successfully.
trainable params: 8,388,608 || all params: 6,746,804,224 || trainable%: 0.1243
-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-409033/4132345508.py in <cell line: 0>()
     54
     55 # Create a DataCollatorForLanguageModeling
--> 56 data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
     57
     58 # Initialize the Trainer

NameError: name 'tokenizer' is not defined

```

Reasoning: The previous code block failed because `tokenizer` was not defined within its scope. I need to re-initialize the `tokenizer` variable by including its loading and configuration code within this block, along with the other re-initialized variables, to ensure the `DataCollatorForLanguageModeling` can be created successfully.

```

from transformers import TrainingArguments, Trainer, DataCollatorForLanguageModeling, AutoModelForCausalLM, BitsAndBytesConfig, AutoTokenizer
from peft import get_peft_model, LoraConfig
import torch

# Re-initialize model_name and tokenizer, as they are used here and might not persist across kernel resets
model_name = "NousResearch/Llama-2-7b-chat-hf"

# 1. Load the pre-trained LLM with 4-bit quantization (re-added from a previous cell)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_use_double_quant=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True,
)

# Re-initialize tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token # Set pad token to eos token for consistency

# 2. Initialize LoRA configuration for PEFT (re-added from a previous cell)
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)

# Apply the PEFT LoRA configuration to the base model
model = get_peft_model(model, lora_config)
print("LoRA model created successfully.")
model.print_trainable_parameters() # Display trainable parameters

# Define TrainingArguments
training_args = TrainingArguments(
    output_dir="./lora_finetuned_model", # Output directory for checkpoints and logs
    num_train_epochs=1,                  # Number of training epochs
    per_device_train_batch_size=4,        # Batch size per GPU/CPU for training
    gradient_accumulation_steps=8,         # Number of updates steps to accumulate before performing a backward/update pass
    optim="paged_adamw_8bit",             # Optimizer type
    learning_rate=2e-4,                   # Learning rate
    logging_steps=10,                     # Log every x updates steps
    save_strategy="epoch",                 # Save checkpoint every epoch
    report_to="none",                     # Disable integrations to avoid errors if not configured

```

```

push_to_hub=False,
remove_unused_columns=False,
fp16=True if torch.cuda.is_available() else False, # Use fp16 for faster training if GPU is available
dataloader_pin_memory=True if torch.cuda.is_available() else False, # Only pin memory if a GPU is available
)

# Create a DataCollatorForLanguageModeling
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

# Initialize the Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    data_collator=data_collator,
)

# Start the training process
print("Starting finetuning...")
trainer.train()
print("Finetuning completed.")

# Optionally, save the fine-tuned model
# trainer.save_model("./lora_finetuned_model_final")

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
 Loading weights: 18% 51/291 [03:11<17:26, 4.36s/it, Materializing param=model.layers.5.mlp.up_proj.weight]

Reasoning: The previous code block failed with a `NameError: name 'tokenizer' is not defined`, even though `tokenizer` was defined within the same code block. This suggests an issue with variable persistence across executions or an unstable environment. To ensure all necessary variables are correctly defined and persist, I will make the training cell completely self-contained by including the dataset loading, tokenization, and label creation steps, in addition to the model and PEFT configuration, and the training setup.

```

from transformers import TrainingArguments, Trainer, DataCollatorForLanguageModeling, AutoModelForCausalLM, BitsAndBytesConfig, AutoTokenizer
from peft import get_peft_model, LoraConfig
import torch
from datasets import load_dataset

# Re-initialize model_name and dataset_name, as they are used here and might not persist across kernel resets
model_name = "NousResearch/Llama-2-7b-chat-hf"
dataset_name = "timdettmers/openassistant-guanaco"

# 1. Load the raw dataset (re-added from a previous cell)
raw_dataset = load_dataset(dataset_name)
print("Dataset loaded successfully:")
print(raw_dataset)

# Re-initialize tokenizer (re-added from a previous cell)
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token # Set pad token to eos token for consistency

# Define a preprocessing function (re-added from a previous cell)
def preprocess_function(examples):
    max_length = 512 # Define a maximum sequence length
    tokenized_inputs = tokenizer(examples["text"], max_length=max_length, truncation=True, padding="max_length")
    return tokenized_inputs

# Apply the preprocessing function to the loaded dataset (re-added from a previous cell)
tokenized_dataset = raw_dataset.map(
    preprocess_function,
    batched=True,
    remove_columns=raw_dataset["train"].column_names # Remove original text columns
)

# Add labels to the tokenized dataset (re-added from a previous cell)
tokenized_dataset = tokenized_dataset.map(
    lambda examples: {'labels': examples['input_ids']},
    batched=True
)

print("\nTokenized dataset with labels:")
print(tokenized_dataset)

# 2. Load the pre-trained LLM with 4-bit quantization (re-added from a previous cell)
bnb_config = BitsAndBytesConfig(

```

```

        load_in_4bit=True,
        bnb_4bit_use_double_quant=True,
        bnb_4bit_quant_type="nf4",
        bnb_4bit_compute_dtype=torch.bfloat16,
    )

    model = AutoModelForCausalLM.from_pretrained(
        model_name,
        quantization_config=bnb_config,
        device_map="auto",
        trust_remote_code=True,
    )

    # 3. Initialize LoRA configuration for PEFT (re-added from a previous cell)
    lora_config = LoraConfig(
        r=16,
        lora_alpha=32,
        target_modules=["q_proj", "v_proj"],
        lora_dropout=0.05,
        bias="none",
        task_type="CAUSAL_LM",
    )

    # Apply the PEFT LoRA configuration to the base model
    model = get_peft_model(model, lora_config)
    print("LoRA model created successfully.")
    model.print_trainable_parameters() # Display trainable parameters

    # Define TrainingArguments
    training_args = TrainingArguments(
        output_dir="./lora_finetuned_model", # Output directory for checkpoints and logs
        num_train_epochs=1, # Number of training epochs
        per_device_train_batch_size=4, # Batch size per GPU/CPU for training
        gradient_accumulation_steps=8, # Number of updates steps to accumulate before performing a backward/update pass
        optim="paged_adamw_8bit", # Optimizer type
        learning_rate=2e-4, # Learning rate
        logging_steps=10, # Log every x updates steps
        save_strategy="epoch", # Save checkpoint every epoch
        report_to="none", # Disable integrations to avoid errors if not configured
        push_to_hub=False,
        remove_unused_columns=False,
        fp16=True if torch.cuda.is_available() else False, # Use fp16 for faster training if GPU is available
        dataloader_pin_memory=True if torch.cuda.is_available() else False, # Only pin memory if a GPU is available
    )

    # Create a DataCollatorForLanguageModeling
    data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

    # Initialize the Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=tokenized_dataset["train"],
        eval_dataset=tokenized_dataset["test"],
        data_collator=data_collator,
    )

    # Start the training process
    print("Starting finetuning...")
    trainer.train()
    print("Finetuning completed.")

    # Optionally, save the fine-tuned model
    # trainer.save_model("./lora_finetuned_model_final")

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
Repo card metadata block was not found. Setting CardData to empty.
WARNING:huggingface_hub.repocard:Repo card metadata block was not found. Setting CardData to empty.
Dataset loaded successfully:
DatasetDict({
  train: Dataset({
    features: ['text'],
    num_rows: 9846
  })
  test: Dataset({
    features: ['text'],
    num_rows: 518
  })
})
Map: 100%                               9846/9846 [00:12<00:00, 787.05 examples/s]
Map: 100%                               518/518 [00:00<00:00, 1102.99 examples/s]
Map: 100%                               9846/9846 [00:04<00:00, 2518.16 examples/s]
Map: 100%                               518/518 [00:00<00:00, 2267.94 examples/s]

Tokenized dataset with labels:
DatasetDict({
  train: Dataset({
    features: ['input_ids', 'attention_mask', 'labels'],
    num_rows: 9846
  })
  test: Dataset({
    features: ['input_ids', 'attention_mask', 'labels'],
    num_rows: 518
  })
})
Loading weights: 100%                   291/291 [18:11<00:00, 3.16s/it, Materializing param=model.norm.weight]
LoRA model created successfully.
trainable params: 8,388,608 || all params: 6,746,804,224 || trainable%: 0.1243
Starting finetuning...

```

Task

Perform memory-optimized LoRA/QLoRA finetuning of the "NousResearch/Llama-2-7b-chat-hf" model using a downsampled version of the "timdettmers/openassistant-guanaco" dataset. Load the model with 4-bit quantization and CPU offloading to save RAM. Tokenize the dataset, truncate sequences to 512 tokens, and select a small subset of 100-200 examples for training. Execute a lightweight finetuning process using the PEFT framework with a small batch size, gradient checkpointing, and a limit of 100 training steps. Finally, verify the process by running a quick inference check on the resulting model and summarize the resource-saving measures used.

Load and Downsample Dataset

Subtask:

Load the 'timdettmers/openassistant-guanaco' dataset and select a small subset of examples to minimize memory usage during training.

Reasoning: I will import the `load_dataset` function, load the specified dataset, shuffle it with a seed for reproducibility, select a subset of 150 examples, and verify the size as instructed.

```

from datasets import load_dataset

# 1. Load the 'timdettmers/openassistant-guanaco' dataset
dataset_name = "timdettmers/openassistant-guanaco"
raw_dataset = load_dataset(dataset_name)

# 2. Extract the 'train' split
train_split = raw_dataset['train']

```

```
# 3. Shuffle and select a small subset of 150 examples for memory efficiency
downsampled_train_dataset = train_split.shuffle(seed=42).select(range(150))
```

```
# 4. Print the number of rows to confirm subset size
print(f"Number of rows in the downsampled training dataset: {len(downsampled_train_dataset)}")
print("Sample data from the downsampled set:")
print(downsampled_train_dataset[0])
```

```
Repo card metadata block was not found. Setting CardData to empty.
WARNING:huggingface_hub.repocard:Repo card metadata block was not found. Setting CardData to empty.
Number of rows in the downsampled training dataset: 150
Sample data from the downsampled set:
{'text': '### Human: Me gradué hace poco de la carrera de medicina ¿Me podrías aconsejar para conseguir rápidamente un puesto de
```

Task

Task

Perform memory-optimized LoRA/QLoRA finetuning of the "NousResearch/Llama-2-7b-chat-hf" model using a downsampled subset (150 samples) of the "timdettmers/openassistant-guanaco" dataset. Load the model with 4-bit quantization (NF4) and double quantization to save VRAM. Initialize the Llama-2-7b tokenizer, preprocess the dataset with a 512-token limit, and create the necessary labels for causal language modeling. Apply LoRA adapters via the PEFT framework, enable gradient checkpointing, and execute a lightweight finetuning process for a maximum of 100 steps using a small batch size. Finally, perform a basic inference check to verify the model is generating text and summarize the resource-saving measures effective in the process.

Tokenize and Prepare Small Dataset

Subtask:

Initialize the tokenizer for Llama-2-7b, preprocess the downsampled dataset (150 samples) with a 512-token limit, and create the labels required for causal language modeling.

Reasoning: I will initialize the tokenizer, define the preprocessing function, and apply it to the downsampled dataset to create the final tokenized data with labels as requested.

```
from transformers import AutoTokenizer

# 1. Load the tokenizer for the model
# model_name is already defined as 'NousResearch/Llama-2-7b-chat-hf'
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token

# 2. Define a preprocessing function
def preprocess_function(examples):
    # Tokenize the 'text' field with a 512-token limit and truncation
    return tokenizer(examples['text'], max_length=512, truncation=True)

# 3. Apply preprocessing to the downsampled dataset
# We remove the original 'text' column to keep only processed features
tokenized_dataset = downsampled_train_dataset.map(
    preprocess_function,
    batched=True,
    remove_columns=downsampled_train_dataset.column_names
)

# 4. Create a 'labels' column by copying 'input_ids'
tokenized_dataset = tokenized_dataset.map(
    lambda examples: {'labels': examples['input_ids']},
    batched=True
)

# 5. Print structure and a sample to verify
print('Processed Dataset Structure:')
print(tokenized_dataset)
print('\nSample Entry (first 10 input_ids):')
sample = tokenized_dataset[0]
print({k: v[:10] if isinstance(v, list) else v for k, v in sample.items()})
```

```

Map: 100%                                150/150 [00:00<00:00, 380.56 examples/s]

Map: 100%                                150/150 [00:00<00:00, 1004.13 examples/s]

Processed Dataset Structure:
Dataset({
  features: ['input_ids', 'attention_mask', 'labels'],
  num_rows: 150
})

Sample Entry (first 10 input_ids):
{'input_ids': [1, 835, 12968, 29901, 2191, 10591, 29948, 20470, 14534, 316], 'attention_mask': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], '

```

✓ Load LLM with Maximum Optimization

Subtask:

Load the pre-trained LLM using 4-bit quantization (NF4) and double quantization to minimize VRAM usage, and prepare it for PEFT adaptation.

Reasoning: I will import the necessary libraries, configure the 4-bit quantization settings with double quantization, load the model with these optimizations, and enable gradient checkpointing to satisfy the memory-optimized loading subtask.

```

from transformers import AutoModelForCausalLM, BitsAndBytesConfig
import torch

# 1. Define BitsAndBytesConfig for maximum VRAM optimization
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16
)

# 2. Load the pre-trained LLM with the defined quantization config
# model_name is 'NousResearch/Llama-2-7b-chat-hf'
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto",
    trust_remote_code=True
)

```